

RAPPORT

AR (MU4IN403) - PROJET :

Implémentation du protocole P2P CHORD

membres:

Xiaoquan LI

Weida LIU

Catalogue

Exercice 1 – Recherche d’une clé	1
Exercice 2 – Calcul des finger tables	1
2.1 Description de l’algorithme :	1
2.2 la justification de la correction:	2
2.3 la justification de la complexité:	3
2.4 code	3
Exercice 3 – Insertion d’un pair	4
3.1 Description de l’algorithme :	4
3.2 la justification de la correction:	5
3.3 la justification de la complexité:	7
3.4 code	8

Exercice 1 – Recherche d'une clé

Veuillez vous référer à EX1.c dans le fichier soumis, qui contient notre solution au premier problème, avec les commentaires appropriés ajoutés.

Exercice 2 – Calcul des finger tables

2.1 Description de l'algorithme :

Dans un réseau en anneau initialisé par un processus de simulation, chaque nœud possède deux attributs : son propre identifiant et l'identifiant du nœud qui lui succède.

Le processus d'exécution de l'algorithme est divisé en deux phases principales :

- 1.initialisation
- 2.la collecte des identifiants
- 3.la construction de la table des empreintes.

2.1.1 Phase d'initialisation

Chaque nœud reçoit un identifiant unique dans la structure en anneau et connaît les identifiants de ses successeurs directs.

2.1.2 Phase de collecte des identifiants.

L'initiateur commence l'exécution de l'algorithme en créant un message contenant son identifiant, en marquant ce message avec le TAG collect et en l'envoyant aux nœuds qui lui succèdent.

Chaque nœud qui reçoit le message de collecte ajoute son propre identifiant au message et transmet le message mis à jour au nœud suivant.

Ce processus se poursuit jusqu'à ce que le nœud initiateur reçoive à nouveau le message marqué comme "collect", qui contient les identifiants de tous les nœuds.

2.1.3 Phase de construction de les finger table.

L'initiateur convertit le message reçu avec le TAG collect, qui contient les identifiants de tous les nœuds, en un message avec le TAG transmit et le renvoie aux nœuds suivants.

Après avoir reçu le message de transmission, chaque nœud utilise les données d'identification contenues dans le message pour construire sa propre finger table .

Une fois la finger table constituée, chaque nœud transmet le message de transmission à son successeur.

Lorsque le nœud initiateur reçoit à nouveau le message de transmission, cela signifie que tous les

nœuds ont construit avec succès leurs finger tables, et le nœud initiateur construit sa propre finger table, et l'ensemble du processus est terminé.

2.2 la justification de la correction:

Afin de prouver l'exactitude de l'algorithme, nous devons prouver deux propriétés de l'algorithme :

1. la sécurité : elle garantit que chaque nœud possède une liste complète de tous les identifiants de nœuds à la fin de l'algorithme et que sa finger table est construite correctement.
2. la vivacité : elle garantit que l'algorithme progresse toujours jusqu'à son terme, que tous les messages sont traités et qu'il n'y a pas de blocage ou d'attente infinie.

2.2.1 Preuve de la sécurité:

Dans la phase COLLECT, lorsque chaque nœud reçoit un message du nœud précédent contenant certains identifiants de nœuds, il ajoute son propre identifiant à la liste et le transmet au nœud suivant.

En raison de la nature structurelle du réseau en anneau, cela garantit que le message sera délivré de manière séquentielle à chaque nœud et qu'il sera finalement renvoyé au nœud d'origine.

Lorsque le nœud d'origine reçoit à nouveau ce message, il contient nécessairement les identifiants de tous les nœuds, ce qui prouve que l'intégrité a été atteinte. Par la suite, chaque nœud construit sa propre finger table sur la base de cette liste, ce qui garantit l'exactitude de la finger tables.

La sécurité de l'algorithme peut donc être prouvée.

2.2.2 Preuve de la vivacité :

Dans cet algorithme, les messages de la phase COLLECT sont transmis séquentiellement d'un nœud à son successeur direct, et comme il s'agit d'une structure en boucle fermée, chaque nœud recevra et traitera le message, et le message ne sera pas bloqué à un certain nœud, ce qui garantit la continuité de la transmission du début à la fin.

En outre, le réseau en anneau garantit que même en cas de défaillance d'un nœud, la transmission du message peut se poursuivre grâce au mécanisme de récupération, ce qui répond à l'exigence d'activité.

Il n'y a pas non plus de dépendances complexes ou de conditions d'attente externes dans la deuxième ou la troisième phase de l'algorithme, ce qui réduit encore la possibilité de blocage.

La vivacité de l'algorithme peut donc être prouvée.

En conclusion, on peut prouver que l'algorithme est complet et actif, et donc qu'il est correct.

2.3 la justification de la complexité:

Tout au long du processus (dans l'hypothèse de N nœuds)

2.3.1 Phase d'initialisation

Dans la première phase, les processus individuels (nœuds individuels) envoient leurs identifiants au processus du simulateur, puis le processus du simulateur renvoie les identifiants des nœuds successeurs aux processus individuels.

Dans ce processus, il y aura N messages envoyés au simulateur et en même temps le simulateur enverra N messages.

La complexité des messages de cette phase est donc $O(n)$.

2.3.2 Phase la collecte des identifiants

Dans la deuxième phase, chaque nœud voudra envoyer un message au nœud suivant jusqu'à ce qu'il soit renvoyé au nœud initiateur, de sorte qu'un total de N messages sera envoyé.

La complexité des messages dans ce processus est donc $O(n)$.

2.3.3 la construction de la table des empreintes phase

Dans la troisième phase, chaque nœud continue à envoyer un message au nœud suivant jusqu'à ce qu'il soit renvoyé au nœud initiateur, soit un total de N messages.

La complexité des messages dans ce processus est donc $O(n)$.

En résumé, la complexité des messages dans l'ensemble du processus est $O(n) + O(n) + O(n) = O(n)$.

$O(n) < O(n^2)$, la complexité de l'algorithme répond donc aux exigences.

2.4 code

Veuillez vous référer à EX2.c dans le fichier soumis, qui contient notre solution au premier problème, avec les commentaires appropriés ajoutés.

Exercice 3 – Insertion d'un pair

3.1 Description de l'algorithme :

3.1.1 Prérequis

Dans un système distribué en anneau implémentant le protocole Chord, chaque nœud maintient une table de routage appelée "table de finger" pour permettre des requêtes efficaces. Le but de cet algorithme est de mettre à jour les données de l'ensemble du système de manière distribuée lors de l'insertion d'un nouveau nœud P_i .

3.1.2 Nœuds impliqués

P_0 : Initialement reçoit l'ID de P_i .

P_{pred} : Le prédécesseur de P_i , qui doit mettre à jour son successeur pour P_i .

P_{succ} : Le successeur de P_i , qui sera enregistré dans P_i .

P_i : Doit configurer sa propre table de finger.

P_n : Autres nœuds, si pointés par la table de finger de P_i , doivent mettre à jour leurs tables de finger inverses ; si leurs tables de finger doivent être mises à jour en raison de l'ajout de P_i , elles doivent également être modifiées.

3.1.3 Objectifs

Déterminer P_{pred} et P_{succ} . P_{pred} peut être trouvé via P_0 en utilisant l'algorithme de recherche, et le successeur enregistré de P_{pred} sera P_{succ} .

Calculer et configurer la table de finger de P_i , et notifier les nœuds concernés dans la table de finger pour mettre à jour leurs tables de finger inverses.

Mettre à jour la table de finger des nœuds associés à P_{succ} pour s'assurer qu'ils pointent vers le bon nœud (P_i ou P_{succ}).

Étapes spécifiques de l'algorithme

Initialisation des messages

Opérations de P_0 :

Après réception de l'ID de P_i , utilise sa table de finger pour localiser le prédécesseur logique (P_{pred}) et le successeur (P_{succ}) de P_i .

Notifie P_i pour mettre à jour son successeur à P_{succ} .

Envoie l'ID de P_i et les informations sur P_{pred} et P_{succ} à P_i et P_{pred} .

Opérations de P_i :

Après avoir reçu les informations de P_0 , configure son successeur à P_{succ} .

Reçoit les informations de la table de finger de P_{pred} et met à jour sa propre table de finger en conséquence.

Envoie un message à tous les nœuds dans sa table de finger pour demander la mise à jour de

leurs tables de finger inverses et attend la confirmation.

Opérations de Ppred :

Après réception de l'ID de Pi, met à jour son successeur à Pi.

Utilise sa table de finger et la distance à Pi pour calculer la table de finger de Pi, et l'envoie à Pi.

Opérations de Psucc :

Après réception de l'ID de Pi, informe tous les nœuds dans sa table de finger inverse pour préparer la mise à jour.

Opérations de Pn :

Suite aux messages reçus, met à jour sa table de finger, ajustant les parties qui pointaient vers Psucc pour pointer vers Pi.

Après chaque mise à jour, envoie un message à Pi et Psucc pour demander la mise à jour de la table de finger inverse.

Après toutes les mises à jour, envoie un message de confirmation à Psucc.

3.1.4 Confirmation de fin

Pi et Psucc, après avoir reçu tous les messages de confirmation nécessaires, envoient chacun un message à P0 pour signaler la fin des modifications.

Après confirmation de toutes les modifications par les nœuds impliqués, le processus d'ajout du nouveau nœud Pi est terminé.

3.2 la justification de la correction:

Afin de prouver l'exactitude de l'algorithme, nous devons prouver deux propriétés de l'algorithme :

1. la sécurité : le système doit conserver un état cohérent, quel que soit le nombre d'opérations effectuées, et chaque opération doit maintenir les contraintes de cohérence du système sans introduire d'erreurs ou de conflits.
2. la vivacité : le système doit être en mesure de répondre aux demandes de manière cohérente sans s'arrêter ou entrer dans une boucle infinie, en veillant à ce que chaque opération soit finalement achevée.

3.2.1 Preuve de la sécurité:

La sécurité garantit que l'état du système reste cohérent à tout moment, c'est-à-dire que les données et l'état de tous les nœuds respectent les contraintes imposées par le système à tout moment. Les contraintes clés pour cet algorithme sont :

La justesse des nœuds successeur et prédécesseur : chaque nœud doit correctement maintenir son successeur et son prédécesseur.

La cohérence de la table Finger : chaque nœud doit correctement pointer vers les nœuds prédéfinis dans sa table Finger, afin de maintenir une performance de recherche efficace.

Preuve :

Mise à jour des nœuds successeur et prédécesseur :

Lorsqu'un nouveau nœud P_i est inséré, P_{pred} et P_{succ} sont explicitement recherchés et confirmés par la conception de l'algorithme. P_i devient le successeur de P_{pred} et P_{succ} devient le successeur de P_i . Cette mise à jour directe garantit la justesse des successeurs et prédécesseurs dans la portée locale de l'insertion.

Comme P_{pred} met à jour son successeur à P_i , et que P_i met à jour son successeur à P_{succ} , cela maintient l'intégrité du cercle, c'est-à-dire que chaque nœud pointe correctement vers son successeur.

Mise à jour de la table Finger :

La table Finger de P_i est calculée et envoyée à P_i par P_{pred} , basée sur la table Finger correcte existante de P_{pred} et les informations de distance avec P_i . Cela garantit que la table Finger de P_i est correcte lors de son intégration au réseau.

Tous les nœuds pointés par la table Finger de P_i reçoivent une demande de mise à jour et ajustent leur propre table Finger inverse, ce qui assure en outre la justesse des chemins de recherche dans le réseau.

La sécurité de l'algorithme peut donc être prouvée.

3.2.2 Preuve de la vivacité :

La vivacité garantit que le système peut continuer à répondre aux demandes sans entrer en blocage ou attente infinie. Les exigences clés de vivacité pour cet algorithme sont :

L'ajout réussi de nouveaux nœuds : un nouveau nœud P_i peut s'ajouter avec succès et complètement au système sans que l'exécution de l'algorithme ne mette le système dans un état non réactif.

L'achèvement de la transmission des informations : toutes les mises à jour nécessaires dues à l'ajout du nouveau nœud doivent être correctement complétées et confirmées.

Preuve :

Pas de blocage :

Dans l'algorithme, chaque opération de nœud est limitée à des étapes prédéfinies et les communications entre les nœuds dépendent unidirectionnellement (par exemple, P_i attend les informations de P_{pred} et P_{succ} , qui ne dépendent pas des entrées de P_i), donc il n'y a pas de conditions d'attente circulaire pour les ressources.

Transmission et confirmation des messages :

Chaque nœud nécessitant une mise à jour de sa table Finger en raison de l'ajout du nouveau nœud P_i envoie un message de confirmation après la mise à jour à P_i ou à P_{succ} . Cela garantit que chaque mise à jour est exécutée et confirmée.

P_i et P_{succ} collectent les messages de confirmation des nœuds concernés, et seulement après la confirmation de toutes les mises à jour nécessaires, ils rapportent l'achèvement à P_0 , garantissant qu'aucune mise à jour n'est omise.

La vivacité de l'algorithme peut donc être prouvée.

En conclusion, on peut prouver que l'algorithme est complet et actif, et donc qu'il est correct.

3.3 la justification de la complexité:

3.3.1 Hypothèses :

Le système comprend au total N nœuds.

La taille de la table de finger de chaque nœud est $\log N$ (ce qui est typique pour le protocole Chord).

Les nœuds P_{pred} et P_{succ} notifient chacun environ $\log N$ nœuds (basé sur leur propre table de finger).

3.3.2 Nombre de messages:

Messages liés à P_0 :

2 messages (à P_{pred} et P_{succ})

1 message (à P_i)

Messages liés à P_i :

1 message (recevant la table de finger de P_{pred})

$\log N$ messages (envoyant des messages à chaque nœud dans sa table de finger pour la mise à jour)

Messages liés à P_{pred} :

1 message (recevant l'ID de P_i)

$\log N$ messages (interrogeant les nœuds de sa table de finger pour établir la table de finger de P_i , puis envoyant celle-ci à P_i)

Messages liés à P_{succ} :

$\log N$ messages (notifiant chaque nœud dans sa table de finger inverse pour la mise à jour)

Messages liés aux P_n :

Chaque P_n envoie 1 message à P_i et 1 message à P_{succ} pour confirmation, supposant que les P_n affectés sont également environ $\log N$, totalisant $2 * \log N$ messages.

En résumé, la complexité moyenne est de $O(3) + O(1) + O(\log n) + O(1) + O(\log n) + O(\log n) = O(\log n)$

$O(\log n) < O(n)$, la complexité de l'algorithme répond donc aux exigences.

3.4 code

Veuillez vous référer à EX3.c dans le fichier soumis, qui contient notre solution au premier problème, avec les commentaires appropriés ajoutés.

Toutefois, en raison de contraintes de temps, nous n'avons en fait achevé que la construction de la table inversée, veuillez donc nous en excuser.